DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 1
Introducción al acceso a datos

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD1 – Introducción al acceso a datos

ÍNDICE

1. Introducción
2. Acceso a datos
    2.1 Ficheros en archivos locales
    2.2 Bases de Datos de acceso por archivos
    2.3 Bases de Datos de tipo Cliente/Servidor
    2.4 Mapeo de los datos (ORM - Object Relational Mapping)
    2.5 Bases de Datos Cliente/Servidor con acceso remoto
    2.6 Bases de Datos Cliente/Servidor Distribuidas
    2.7 Virtualización
    2.8 Servicios de BD en la nube (on cloud)
    2.9 Bases de Datos noSQL
    2.10 Servidores Web como pasarela para el acceso a datos
    2.11 BIG DATA
    2.12 Resumen de Tecnologías a estudiar
3. Ficheros
4. Base de datos
    4.1.- Introducción
    4.2.- Bases de datos relacionales (BDR)
    4.3.- Bases de datos orientadas a objetos (BDOO)
    4.4.- Comparativa entre BDR y BDOO
    4.5.- Bases de datos objeto-relacionales (BDOR)
5. Acceso a BD mediante conectores
6. Mapeo objeto relacional (ORM)
    6.1.- Capa de persistencia y framework de mapeo
7. Bases de datos XML
8. Bases de datos Documentales
9. Desarrollo de componentes
    9.1.- Definición de componente
    9.2.- JavaBeans

1. Introducción

Iniciamos esta primera unidad del módulo Acceso a datos, en el que veremos la gran variedad de métodos de acceso a datos que tenemos en el panorama actual.

Los contenidos del módulo son eminentemente prácticos, y los ejemplos estarán basados en Java. Utilizaremos como entorno de programación NetBeans, por coherencia con otros módulos del ciclo y por ser ambos gratuitos. Además, NetBeans es un entorno muy potente y fácil de usar, si lo comparamos con otros entornos de desarrollo.

Pero, ¿a qué nos referimos cuando hablamos de acceso a datos en una aplicación informática?

Podemos afirmar que en la inmensa mayoría de aplicaciones informáticas se pueden diferenciar, a grandes rasgos, en dos partes:

- Por un lado, el **programa** propiamente dicho, que realiza las operaciones deseadas con los datos necesarios.
- Por otro lado, los **datos** con los que opera le programa. Esos datos pueden ser obtenidos por el programa mediante diversos métodos: leídos mediante teclado, escaneados, leídos de algún soporte de almacenamiento secundario, etc.

En la mayoría de los casos, cuando programamos, nos interesa que el programa guarde los datos que le hemos introducido, o los resultados que dicho programa haya obtenido, de manera que si el programa termina su ejecución, los datos no se pierdan y puedan ser recuperados posteriormente, es decir, persistan. Una forma tradicional de hacer esto es mediante la utilización de ficheros o de bases de datos que se guardarán en un dispositivo de memoria no volátil (normalmente un disco).

El almacenamiento en memoria RAM, mediante variables o arrays, es temporal, los datos se pierden cuando el programa termina. Quizás te habrá pasado alguna vez que, debido a un apagón eléctrico, has perdido el trabajo que estabas haciendo, que todavía no habías grabado. **Los datos que se guardan en almacenamiento secundario, como ficheros o bases de datos, se denominan datos persistentes**, porque existen, o persisten más allá de la ejecución de la aplicación.

Ese almacenamiento secundario de datos que acabamos de mencionar, habitualmente suele consistir en una base de datos relacional, si bien, a veces, hay otros métodos de almacenamiento, y por tanto, métodos de acceso a esos datos. De conocer esos tipos de almacenamiento y cómo acceder a ellos es de lo que trata este módulo.

En esta unidad inicial, vas a ver una panorámica de los diversos métodos de persistencia que encontramos en el mercado.

2. Acceso a datos

El acceso a datos ha sufrido el últimos años una gran evolución. Según han ido avanzando las tecnologías y aumentando las necesidades de las empresas, se han desarrollado tecnologías para darles solución.

Veamos un breve repaso de las tecnologías usadas.

2.1 Ficheros en archivos locales

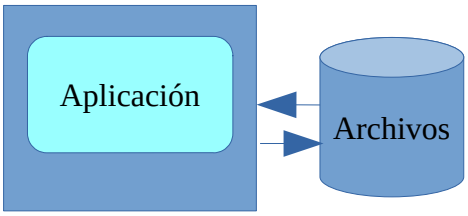
Al principio, en los primeros tiempos de la informática, los datos se guardaban en ficheros convencionales.

Necesidades:

- Almacenar datos que sean accesibles al volver a encender el sistema

Los datos se guardan en archivos del propio ordenador donde se encuentra la aplicación.

Conexión de acceso a datos:



Ejemplos de uso actual:

- Para guardar datos de configuraciones
- Para realizar exportaciones de datos entre diferentes sistemas
- Para almacenar datos en pequeños dispositivos como móviles o tabletas

Ejemplos de Tecnologías:

- JSON, CSV, XML

2.2 Bases de Datos de acceso por archivos

Con el tiempo, y la experiencia de trabajar con dichos ficheros, se observaron los inconvenientes de los ficheros, y para intentar solucionar los inconvenientes que se observaron surgieron las **bases de datos**, que entre otras ventajas permitían:

- Eliminar el problema de la **información redundante**.
- Eliminar **información inconsistente**.
- Globalizar o **centralizar la información**.
- Garantizar el **mantenimiento de la integridad** en la información. Únicamente se almacena la información correcta.
- **Independencia de datos,** que implica una separación entre programas y datos, es decir, se pueden hacer cambios en la información que contiene la base de datos, o tener acceso a la base de datos de diferente manera, sin tener que hacer cambios en las aplicaciones o en los programas.

Hay diversas bases de datos para gestionar la persistencia de los datos:

- BD Relacionales (BDR)
- BD Orientadas a objetos (BDOO)
- BD Objeto-relacionales (BDOR)
- BD Documentales (BDD)

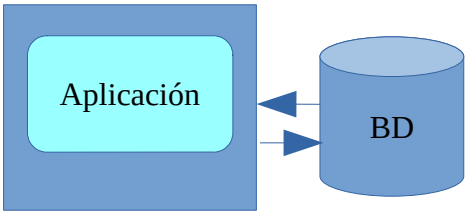
Nuevas necesidades:

- Aumentan los datos y se necesita más velocidad de respuesta

Se desarrollan tecnologías complejas de almacenamiento e indexación que aumentan la velocidad a la hora de realizar consultas.

La Base de Datos suele ser un archivo que contiene todos los datos, o bien una carpeta que contiene los archivos para datos y para índices.

Conexión de acceso a datos:



Ejemplos de uso actual:

- Para pequeños sistemas de un solo usuario

Ejemplos de Tecnologías:

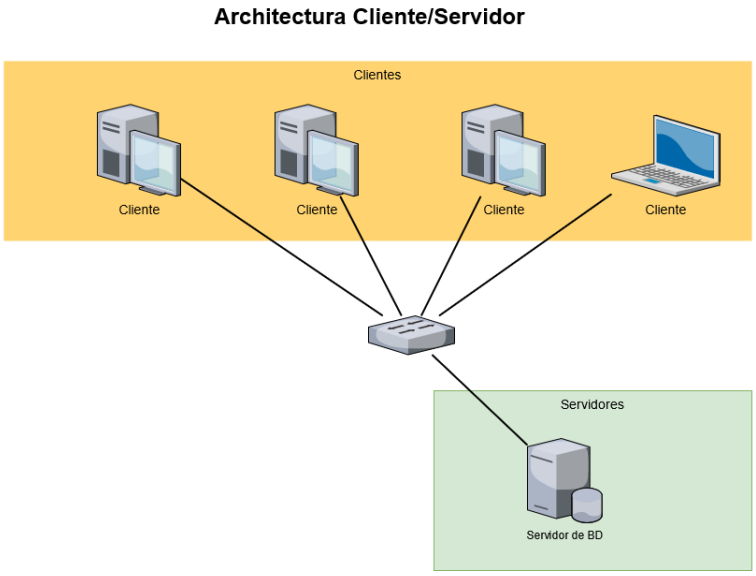
- SQLite, Microsoft Access, H2

2.3 Bases de Datos de tipo Cliente/Servidor

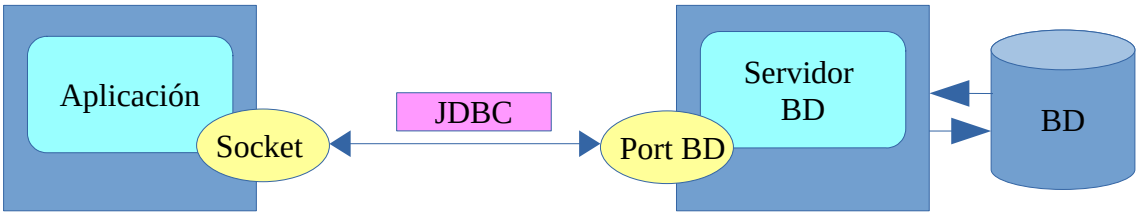
Nuevas necesidades:

- Se necesita acceso de varios usuarios simultáneamente a los datos
- Se separa la aplicación de la base de datos, permitiendo que cada una esté en ordenador diferente. La aplicación será el Cliente y la Base de Datos el servidor.
- Se accede utilizando redes locales, y lo más usado es TCP/IP.

Aunque hay muchos tipos de tecnologías de BD, las más usadas son las de tipo relacional.



Conexión de acceso a datos:



Ejemplos de Tecnologías:

- MySQL, Oracle, SQL Server, PostgreSQL
- Acceso mediante driver JDBC a un puerto TCP

2.4 Mapeo de los datos (ORM - Object Relational Mapping)

Nuevas necesidades:

- Simplificar la programación a la hora de obtener y actualizar los datos en la BD al tenerlos almacenarlos en RAM en objetos.

En la programación Orientada a Objetos se realiza un mapeo de los datos almacenados a los objetos en memoria. Este proceso es un proceso que depende directamente de la estructura del driver y la BD.

Para simplificar este proceso surgen tecnologías que realizan este traspaso de forma automática y transparente, permitiendo incluso su exportación/transformación a formatos estándar como JSON o XML.

Ejemplos de Tecnologías:

- JPA
- Hibernate, TopLink, EclipseLink, OpenJPA, ObjectDB

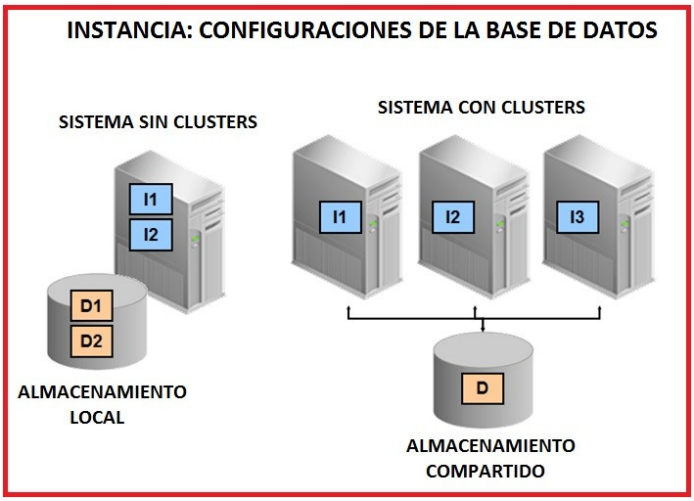
2.5 Bases de Datos Cliente/Servidor con acceso remoto

Nuevas necesidades:

- Se disponen de varias oficinas y se crea un centro de proceso de datos compartido para todas.
- El acceso a los datos se realiza utilizando Internet a gran velocidad

Las grandes corporaciones diseñan diferentes estructuras de centros de procesos de datos (**CPD** o **Data Center).** Un **Data Center** es un espacio físico especialmente preparado como entorno operativo para la actividad del alojamiento de datos donde se encuentran servidores, equipos informáticos, equipos de red, y sistemas de almacenamiento.

Los Data Center estructuran sus servidores de datos en **clusters,** es decir, creando una arquitectura en la que tenemos varios equipos con parte de los datos del usuario trabajando al unísono como un solo sistema. La arquitectura de un cluster de base de datos viene definida por la manera en que se almacenan los datos en cada nodo.



Normalmente, son instalaciones (edificios o salas) con capacidad para alojar cientos de servidores que cuentan con control de acceso, dispositivos de seguridad, detección de humos en caso de incendio con extractores de aire y sistema de extinción, conectividad, suministro eléctrico redundado y continuado, así como una climatización óptima (temperatura y humedad).

Tenemos dos tipos principalmente:

- **Data Warehouse** - es un almacén de datos electrónico donde generalmente una empresa u organización mantiene una gran cantidad de información. Los datos de un data warehouse deben almacenarse de forma segura, fiable, fácil de recuperar y fácil de administrar, además los datos están filtrados y estructurados pues ya han sido procesados para una finalidad concreta.
- **Data Lake** - es un enorme conjunto de datos en bruto cuya finalidad no se ha definido todavía.

2.6 Bases de Datos Cliente/Servidor Distribuidas

Nuevas necesidades:

- La globalización crea la necesidad de distribuir los datos para geolocalizarlos más cerca de los lugares donde se usan, distribuyendo los datos y proporcionando sistemas de redundancia para garantizar el servicio.

Una **Base de Datos Distribuida** (BDD) es un conjunto de múltiples bases de datos lógicamente relacionadas las cuales se encuentran distribuidas en diferentes espacios lógicos y geográficos (por ejemplo, un servidor ejecutándose en 2 máquinas virtuales) e interconectados por una red de comunicaciones. Dichas BDD tienen la capacidad de realizar procesamientos autónomos, estos permiten realizar operaciones locales o distribuidas. Esto permite crear Data Warehouse y Data Lakes interconectados entre sí formando **Data Centers Distribuidos**.

2.7 Virtualización

Nuevas necesidades:

- La gran cantidad de componentes que se configuran en un sistema informático obliga a separarlos para simplificar la configuración y el mantenimiento

La **virtualización** tanto de sistemas operativos completos como la de contenedores ha permitido en los últimos años disponer de instalaciones de microservicios en una empresa que interconectados completan un sistema informático con mucha más flexibilidad y escalabilidad.

Ejemplos de Tecnologías:

- VirtualBox, Microsoft Hyper-V, VMWare, KVM, XEN
- Docker, Kubernetes

2.8 Servicios de BD en la nube (on cloud)

Nuevas necesidades:

- Las necesidades son variables en el tiempo y muy costosas, por lo que cada vez más se usan instalaciones en la nube (**on cloud)**, en detrimento de las creadas en los locales de cada empresa (**on premise**).
- Se busca flexibilidad, escalabilidad y amortización de las inversiones, es decir, pagar por el uso de lo que se necesita.

Las empresas buscan aprovechar los últimos avances tecnológicos, es decir:

- Velocidad en las comunicaciones a través de internet
- Gran capacidad de almacenamiento a bajo precio
- Grandes centros de procesamiento de datos
- Tecnologías de bases de datos distribuidas
- Virtualización
- Servicios en la nube
- ...

Utilizando todas estas tecnologías surgen los **Edge Data Centers**, es decir, sistemas de informáticos descentralizados que llevan la potencia informática directamente al punto de generación de los datos. Se encuentran cerca de las fuentes de datos y, por tanto, permiten una respuesta del procesamiento de los datos de una manera más rápida ya que reducen el tiempo de latencia en la conexión.

Además, también están conectados a centros de datos en el cloud, ya que existe un procesamiento posterior.

Ejemplos de Tecnologías:

- Microsoft Azure, Amazon Web Services, Google Cloud

2.9 Bases de Datos noSQL

Nuevas necesidades:

- Procesamiento de gran cantidad de datos
- Almacenar los datos en estructuras más flexibles, es decir, no estar sujetos a diseños de tablas como en la BD relacionales
- Realizar un procesamientos de datos más eficiente

Las características principales de las Bases de Datos NoSQL son:

- SQL no es el lenguaje de consulta/modificación de datos
- Los datos no tienen que almacenarse en tablas
- Suelen disponer de arquitectura distribuida y se accede mediante conexión Cliente/Servidor

- Son más eficientes en el procesamiento de los datos que las BD relacionales, en detrimento de perder coherencia en datos o tener redundancia.
- No hay una estandarización de este tipo de BD
- Se usan cuando no hay necesidad de que los datos sean consistentes, no se dispone de una estructura fija para almacenar la información y se realiza una consulta/lectura intensiva de grandes cantidades de datos.

Son utilizadas por empresas como Amazon, Facebook o Google.

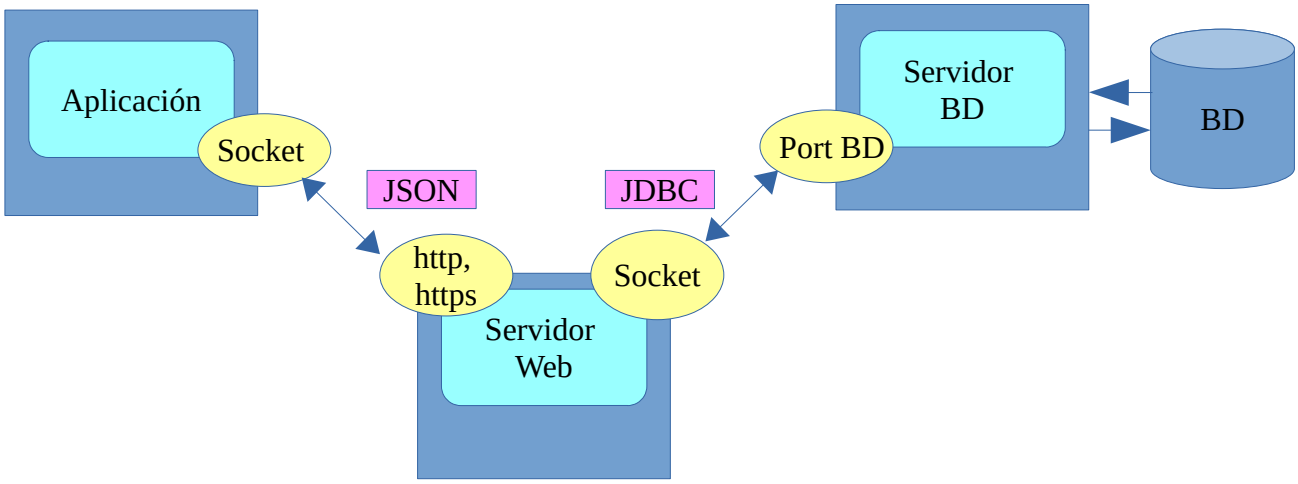
Ejemplos de Tecnologías:
- Cassandra, Redis, MongoDB o CouchDB
- Acceso mediante driver JDBC a un puerto TCP

2.10 Servidores Web como pasarela para el acceso a datos

Necesidades:
- Acceso a los datos independiente de la tecnología

Cada servidor de BD prepara su driver JDBC para el acceso a datos y obliga a los programadores a estudiar su API para poder realizar la aplicación. La complejidad de las aplicaciones y su mantenimiento en caso de cambios de tecnologías disminuye notablemente si en vez de usar JDBC como sistema de acceso a los servidores de BD, instalamos un servidor Web que realice las operaciones de acceso lo que nos permite realizar una comunicación con un formato estándar como JSON o XML.

Conexión de acceso a datos:



Ejemplos de Tecnologías:
- SOAP (1998)
- API REST (2000)
- GraphQL (2015)

2.11 BIG DATA

Nuevas necesidades:

- Almacenar y procesar la gran cantidad de datos que se generan mediante el uso de internet tanto en ordenadores como en dispositivos móviles, incluso en los nuevos dispositivos de Internet de las Cosas (IoT - Internet of Things).

Las características del BIG DATA son varias:

- VOLUMEN: se procesan gran cantidad de datos
- VELOCIDAD: los datos se generan incluso en tiempo real y se necesita una gran capacidad de procesamiento
- VARIEDAD: los datos llegan de diferentes fuentes y en diferentes formatos
- VERACIDAD: los datos deben ser útiles y veraces, es decir, que puedan contrastarse
- VIABILIDAD: los datos deben obtenerse sin un gran costo de inversión
- VISUALIZACIÓN: la información obtenida se debe poder visualizar de forma clara, es decir, en formatos estándar y con gráficos.
- VALOR: la información obtenida debe servir a la empresa a tomar decisiones para mejorar el negocio o el servicio

BIG DATA - Presentación
https://www.youtube.com/watch?v=w4vsFKMO7XA

2.12 Resumen de Tecnologías a estudiar

Las tecnologías que veremos durante el módulo serán:

<table>
<tr><th>PROGRAMACIÓN APLICACIONES</th><th>PROGRAMACIÓN DE SERVIDORES</th><th>SERVIDORES DE BASES DE DATOS</th></tr>
<tr><td rowspan="3">- Java
- POO
- JDBC
- ORM
- Componentes
- Formatos: JSON, XML, CSV</td><td>- App Web
- API REST</td><td>- MySQL
- Oracle
- SQL Server
- MongoDB</td></tr>
<tr><td colspan="2">VIRTUALIZACIÓN</td></tr>
<tr><td colspan="2">Microsoft Azure</td></tr>
</table>

3. Ficheros

Hay que tener en cuenta, que los ficheros en sí, para grabar datos en general no se suelen usar. Se suelen usar en intercambio de datos y en el almacenamiento de configuraciones o parametrizaciones.

Uno de los formatos más antiguos es **CSV** (del inglés Comma-Separated Values). Son un tipo de documento en formato abierto sencillo para representar datos en forma de tabla, en las que las columnas se separan generalmente por comas y las filas por saltos de línea.

Pero, también se usan ficheros que guardan datos siguiendo un patrón o estructura bien definida, como por ejemplo **XML**. Éstos son archivos de texto que por consiguiente no necesitan un software propietario para ser interpretados, como ocurre con la mayoría de los archivos binarios.

Otro formato es el **JSON**, cuya ventaja sobre XML como formato de intercambio de datos es que resulta mucho más sencillo escribir un analizador sintáctico (parser) para él. Últimamente se ha convertido en el formato estándar más utilizado en el acceso a datos.

También debemos tener en cuenta, que algunas bases de datos relativamente modernas, guardan sus datos empleando ficheros XML o JSON.

Por eso, en muchas ocasiones se recurre a utilizar este tipo de soluciones, el uso de ficheros en vez de bases de datos, cuando se necesita intercambiar información a través de varias plataformas de hardware o de software, o de varias aplicaciones. A veces se exporta de una base de datos a ficheros a XML, JSON o CSV para trasladar la información a otra base de datos que leerá esos ficheros.

4. Bases de Datos

Quizás hayas estudiado ya el módulo de bases de datos. Tanto si es así como si no, recordamos aquí qué es un sistema de bases de datos. Es:

- Un sistema de información orientado hacia los datos, que pretende recuperar y almacenar la información de manera eficiente y cómoda.
- Surge en un intento de resolver las dificultades del procesamiento tradicional de datos, teniendo en cuenta que los datos suelen ser independientes de las aplicaciones.

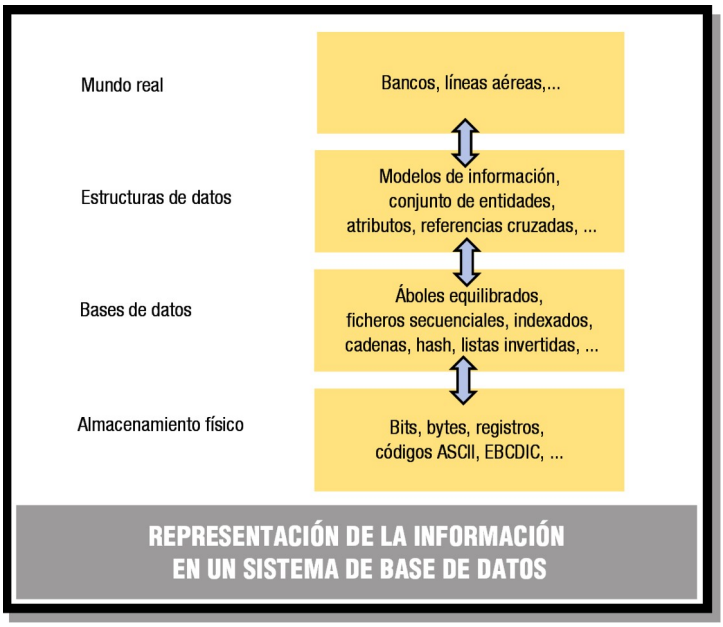
4.1.- Introducción

Las ventajas que aportan los sistemas de bases de datos respecto a los sistemas de archivos convencionales son:

- **Independencia** de los datos respecto de los procedimientos. El usuario tiene una visión abstracta de los datos, sin necesidad de ningún conocimiento sobre la implementación de los ficheros de datos, índices, etc. Esto supone un gran ahorro en los costes de programación, de forma que la modificación de la estructura de los datos no suponga un cambio en los programas y viceversa. Sin ella, el mantenimiento de la base de datos ocuparía el 50% de los recursos humanos dedicados al desarrollo de cualquier aplicación.
- Disminución de las **redundancias** y en consecuencia,
- Disminución de la posibilidad de que se produzca **inconsistencia** de datos.
- Mayor **integridad** de los datos.
- Mayor **disponibilidad** de los datos.
- Mayor **seguridad** de los datos.
- Mayor **privacidad** de los datos.
- Mayor **eficiencia** en la recogida, codificación y entrada en el sistema.
- Lo que se suele denominar interfaz con el pasado y futuro: una base de datos debe estar abierta a reconocer información organizada físicamente por otro software.
- **Compartición** de los datos. Los datos deben poder ser accedidos por varios usuarios simultáneamente, teniendo previstos procedimientos para salvaguardar la integridad de los mismos.

Podemos afirmar generalizando, que se usa un sistema de ficheros convencional cuando la cantidad de datos a guardar es tan reducida que no justifica las ventajas del uso de los sistemas de bases de datos.

Por ejemplo, para guardar los datos del resultado de la instalación de un programa, usamos un fichero de texto, no se guardan los datos en una base de datos.



4.2.- Bases de datos relacionales (BDR)

Id de cliente	Nombre contac	Apellidos conta	Dirección fact.	Ciudad	Código postal	Nº de teléfono
1	Antonio	López Cruces	C/ Mayor 112	Madrid	29090	91000112
2	Feliciano	Gómez Cuesta	Plaza de Arriba	Barakaldo	10022	92121212
3	José Antonio	Sobrino Lara	C/ Zirnonita 3	Murcia	30001	96800002

Probablemente habrás cursado ya el módulo de bases de datos, si es así, e incluso si no lo es, quizás sepas que el **modelo de bases de datos relacional** fue propuesto en 1969 por Edgar Codd.

El propósito del modelo relacional es proporcionar un método declarativo para especificar datos y consultas. Así, en el diseño de la base de datos establecemos qué información contendrá dicha base de datos, luego recuperaremos la información que queramos, y dejamos al software del sistema gestor de la base de datos que se ocupe de: describir las estructuras de datos para almacenarlos, y gestionar los procedimientos de recuperación para obtener las consultas deseadas.

Las bases de datos relacionales son adecuadas para manejar grandes cantidades de datos, compartir datos entre programas, realizar búsquedas rápidas, etc. Pero tienen como desventaja fundamental que no presentan un buen modelo de las relaciones entre los datos, ya que todo se representa como tablas bidimensionales, o sea, en filas y columnas.

Podemos decir de las bases de datos relacionales que:

- Están muy extendidas.
- Son muy robustas.
- Permiten interoperabilidad entre aplicaciones.
- Permiten una forma de compartir datos entre aplicaciones.
- Son el común denominador de muchos sistemas y tecnologías. Una base de datos puede ser utilizada en programas realizados en Java, o en C++, etc.

Otros modelos tradicionales

Cuando se estudian las bases de datos relacionales, se suelen mencionar también los modelos **jerárquico** y **en red**. Algunos sistemas antiguos utilizan todavía estas arquitecturas, en centros de datos donde se manejan grandes volúmenes de información y la complejidad de los sistemas hace prohibitivo el coste que supondría migrar a sistemas que utilizaran otro modelo como el relacional.

El modelo relacional fue el primer modelo de bases de datos definido de manera formal. Posteriormente, se formularon modelos informales para describir bases de datos jerárquicas (el modelo jerárquico) y bases de datos en red (el modelo en red). Ambas, las bases de datos jerárquicas y en red existían antes que las bases de datos relacionales, pero fueron descritas como modelo después de que el modelo relacional fuera definido.

Bases de datos relacionales (BDR)
https://db-engines.com/en/ranking/relational+dbms

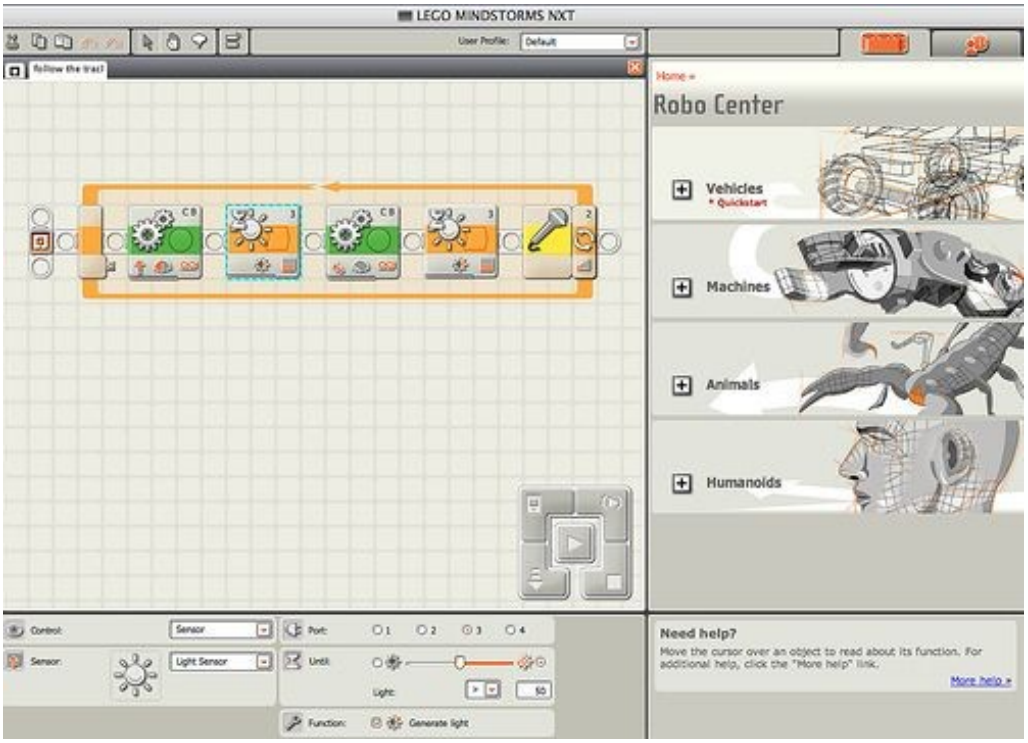
4.3.- Bases de datos orientadas a objetos (BDOO)

El origen de las Bases de datos orientadas a objetos (en adelante: BDOO) se debe básicamente a las siguientes razones:

- La existencia de problemas al representar cierta información y modelar ciertos aspectos del mundo real. Los modelos clásicos permiten representar gran cantidad de datos, pero las operaciones y representaciones que se pueden realizar sobre ellos son bastante simples.
- Pasar del modelo de objetos al modelo relacional, para almacenar la información, genera dificultades que en el caso de las BDOO no surgen, ya que el modelo es el mismo. Es decir, los datos de los programas escritos en lenguaje orientado a objetos se pueden almacenar directamente, sin conversión alguna, en las BDOO.

Los sistemas de bases de datos orientadas a objetos soportan un **modelo de objetos puro**, ya que no están basados en extensiones de otros modelos más clásicos como el relacional.

Por ello, una característica general es que **el lenguaje de programación y el esquema de la base de datos utilizan las mismas definiciones de tipos**.



El acceso a los datos puede ser más rápido con las bases de datos orientadas a objetos que con las bases de datos tradicionales porque no hay necesidad de utilizar las uniones o joins, que si se necesitan en los esquemas relacionales tabulares. Esto se debe a que un objeto puede recuperarse directamente sin una búsqueda, simplemente siguiendo punteros.

Cada vez más, las necesidades de las aplicaciones actuales con respecto a las bases de datos son:

- Soporte para objetos complejos y datos multimedia.
- Jerarquías de objetos o tipos y herencia.
- Gestión de versiones.
- Modelos extensibles mediante tipos de datos definidos por el usuario.
- Integración de los datos con sus procedimientos asociados.
- Manipulación navegacional (en vez de declarativa) y de conjunto de registros.
- Interconexión e interoperabilidad.

Manifiesto de Malcom Atkinson

Características obligatorias de una BD Orientada a Objetos:

1) Deben soportarse objetos complejos
2) Deben soportarse mecanismos de identidad de los objetos
3) Debe soportarse la encapsulación
4) Deben soportarse tipos y clases
5) Los tipos o clases deben ser capaces de heredar de sus ancestros
6) Debe soportarse sobrecarga, sobreescritura y enlace dinámico
7) El DML debe ser computacionalmente completo
8) El conjunto de todos los tipos de datos debe ser ampliable
9) Debe proporcionarse persistencia a los datos
10) El SGBD debe ser capaz de gestionar BD de gran tamaño
11) El SGBD debe soportar a usuarios concurrentes
12) El SGBD debe ser capaz de recuperarse de fallos de hardware y software
13) El SGBD debe proporcionar una forma simple de consultar los datos

Características opcionales de una BD Orientada a Objetos:

1) Herencia múltiple
2) Comprobación de tipos e inferencia de tipos
3) Sistema de BD distribuidores
4) Soporte de versiones

Bases de datos orientadas a objetos (BDOO)
https://db-engines.com/en/ranking/object+oriented+dbms

4.4.- Comparativa entre bases de datos relacionales y orientadas a objetos

En numerosos bancos de pruebas realizados para comparar los sistemas de bases de datos orientados a objetos (ODBMS) y los sistemas de bases de datos relacionales (RDBMS), se ha mostrado que los ODBMS pueden ser superiores para ciertos tipos de tareas.

La explicación a esto es que muchas operaciones se realizan utilizando interfaces navegacionales más que declarativos, y el acceso a datos navegacional es normalmente implementado muy eficientemente.

Un buen argumento a favor de las bases de datos orientadas a objetos es la transparencia (no ensucia la construcción de una aplicación). El desarrollador así se debe preocupar de los objetos de su aplicación, en lugar de cómo los debe almacenar y recuperar de un medio físico.

Podemos decir que las ventajas de un SGBDOO frente a las relacionales son:

- Permiten mayor **capacidad de modelado**. El modelado de datos orientado a objetos permite modelar el mundo real de una manera mucho más fiel. Esto se debe a:
  - un objeto permite encapsular tanto un estado como un comportamiento
  - un objeto puede almacenar todas las relaciones que tenga con otros objetos
  - los objetos pueden agruparse para formar objetos complejos (herencia).
- **Extensibilidad**, debido a que:
  - Se pueden construir nuevos tipos de datos a partir de los ya existentes.
  - Podemos agrupar propiedades comunes de diversas clases e incluirlas en una superclase, lo que reduce la redundancia.
  - Tenemos reusabilidad de clases, lo que repercute en una mayor facilidad de mantenimiento y un menor tiempo de desarrollo.
- Disposición de un **lenguaje de consulta más expresivo**. El acceso navegacional desde un objeto al siguiente es la forma más común de acceso a datos en un sistema gestor orientado a objetos. Mientras que SQL utiliza el acceso declarativo. El acceso navegacional es más adecuado para gestionar operaciones tales como consultas recursivas, etc.
- **Adaptación a aplicaciones** avanzadas de base de datos. Hay muchas áreas en las que las bases de datos relacionales no han tenido excesivo éxito, como es el caso de en sistemas de diseño CAD, CASE, sistemas multimedia, etc. en los que las capacidades de modelado de los SGBDOO han hecho que esos sistemas sí resulten efectivos para este tipo de aplicaciones.
- **Prestaciones**. Los sistemas gestores de bases de datos orientadas a objetos proporcionan mejoras significativas de rendimiento con respecto a los SGBD relacionales. Aunque hay autores, que han argumentado que los bancos de prueba, usados en dichas pruebas, están dirigidos a aplicaciones de ingeniería donde los SGBDOO son más adecuados. También está demostrado, que los SGBDR tienen un rendimiento mejor que los SGBDOO en las aplicaciones tradicionales de bases de datos como el procesamiento de transacciones en línea (OLTP).
- **Reglas de acceso**. En las bases de datos relacionales, a los atributos se accede y se modifican a través de operadores relacionales predefinidos. En las orientadas a objetos se procede mediante las interfaces que se creen a tal efecto de las clases. Desde este punto de vista, los sistemas orientados a objetos dan independencia a cada objeto que el sistema relacional no permite.
- **Clave**. En el modelo relacional, las claves primarias generalmente tienen una forma representable en texto, sin embargo los objetos no necesitan una representación visible del identificador.

Desventajas de las bases de datos orientadas a objetos frente a las relacionales.

Como desventajas o puntos débiles de las BDOO respecto a las relacionales podemos mencionar:

- La **reticencia del mercado**, tanto para desarrolladores como usuarios, a este tipo de bases de datos.
- **Carencia de un modelo de datos universal**. No hay ningún modelo de datos que esté universalmente aceptado para los SGBDOO y la mayoría de los modelos carecen de una base teórica. El modelo de objetos aún no tiene una teoría matemática coherente que le sirva de base.
- **Carencia de experiencia**. Al ser una tecnología relativamente nueva, todavía no se dispone del nivel de experiencia del que se dispone para los sistemas relacionales.
- Panorama actual. Tanto los **sistemas gestores de bases de datos como los sistemas gestores de bases de datos objeto-relacionales están muy extendidos**. SQL es un estándar aprobado y ODBC y JDBC son estándares de facto. Además, el modelo relacional tiene una sólida base teórica y los productos relacionales disponen de muchas herramientas de soporte que sirven tanto para desarrolladores como para usuarios finales.
- **Dificultades en optimización**. La optimización de consultas necesita realizar una compresión de la implementación de los objetos, para poder acceder a la base de datos de manera eficiente. Sin embargo, esto compromete el concepto de encapsulación.

4.5.- Bases de datos objeto-relacionales (BDOR)

Entendemos Base de Datos Objeto Relacional (BDOR), una base de datos que ha evolucionado desde el modelo relacional a otro extendido que incorpora conceptos del paradigma orientado a objetos. Por tanto, un Sistema de Gestión Objeto-Relacional (SGBDOR) contiene ambas tecnologías: relacional y de objetos.

En una Base de Datos Objeto Relacional el diseñador puede crear sus propios tipos de datos y crear métodos para esos tipos de datos.

Por ejemplo, podríamos definir un tipo de la siguiente manera:

```
CREATE TYPE persona_t AS OBJECT (
     nif VARCHAR2(9),
     nombre VARCHAR2(30),
     direccion VARCHAR2(40),
     telefono VARCHAR2(15),
     fecha_nac DATE
);
```

Por poner un ejemplo, si consideramos la base de datos Oracle, debido a los requerimientos de las nuevas aplicaciones, el sistema de gestión de bases de datos relacional desde versión 8i fue extendido con conceptos del modelo de bases de datos orientadas a objetos. De esta manera, aunque las estructuras de datos que se utilizan para almacenar la información siguen siendo tablas, los diseñadores pueden utilizar muchos de los mecanismos de orientación a objetos para definir y acceder a los datos. Se reconoce el concepto de objetos, de tal manera que un objeto tiene un tipo, se almacena en cierta fila de cierta tabla y tiene un identificador único (OID). Estos identificadores se pueden utilizar para referenciar a otros objetos y así representar relaciones de asociación y de agregación.

La ventaja de este tipo de base de datos es que los usuarios pueden pasar sus aplicaciones actuales sobre bases de datos relaciones este nuevo modelo sin tener que reescribirlas. Más tarde, se pueden ir adaptando las aplicaciones y bases de datos para que utilicen las funciones orientadas a objetos.

Con las Bases de Datos Objeto-Relacional se amplía el modelo relacional destacando las siguientes aportaciones:

- Se aumentan la variedad en los tipos de datos,
  - se pueden crear nuevos tipos de datos que permitan construir aplicaciones complejas con una gran riqueza de dominios. Se soportan tipos complejos como: registros, conjuntos, referencias, listas, pilas, colas y vectores.
- Hay extensiones en el control de la Semántica de datos Objeto-Relacionales:
  - Se pueden crear procedimientos almacenados y funciones que tengan un código en algún lenguaje de programación, como por ejemplo: SQL, Java, C, etc.
- Se pueden compartir varias librerías de clases ya existentes, esto es lo que conocemos como reusabilidad.

Como productos comerciales de bases de Datos Objeto-Relacional podemos destacar:

- DB2 Universal Database de IBM (International Business Machines).
- Universal Server de Informix (ahora de IBM),
- INGRES II, de Computer Associates.
- ORACLE de Oracle Corporation,
- SyBASE

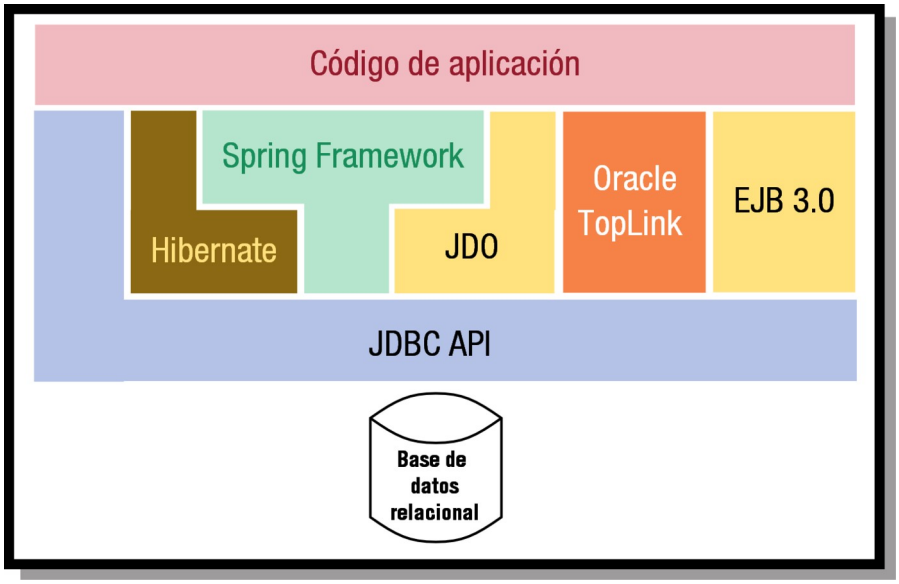
Como productos de código abierto, destacamos PostGreSQL.

Tipos de Bases de datos
https://db-engines.com/en/ranking

5.- Acceso a bases de datos mediante conectores

Para gestionar la información de las bases de datos hay unos estándares que facilitan a los programas informáticos manipular la información almacenada en ellas.

En Java existe un API basado en estos estándares que permite al desarrollar aplicaciones, que se pueda acceder a bases de datos relacionales: JDBC (Java Database Connectivity, conectividad de bases de datos de Java).



La mayoría de las aplicaciones importantes de una empresa están respaldadas por una arquitectura normalizada y optimizada de bases de datos relacionales. Tradicionalmente, dichas aplicaciones están basadas en sentencias SQL con las cuales se gestionan todos los datos que manejan.

Este modelo continúa teniendo una gran importancia estratégica y es la base para el continuo crecimiento del mapeo Objeto-Relacional (O/R) y está asociado a los mecanismos de persistencia.

Un **driver JDBC** es un componente software que posibilita a una aplicación Java interaccionar con una base de datos.

El API JDBC define interfaces y clases para escribir aplicaciones de bases de datos en Java realizando conexiones de base de datos.

Mediante JDBC el programador puede enviar sentencias SQL, y PL/SQL a una base de datos relacional. JDBC permite embeber SQL dentro de código Java.

La ilustración muestra una representación de los diferentes mecanismos de mapeo O/R y cómo se relacionan con el código de la aplicación y con los recursos de datos

relacionados. Se observa claramente la función crítica que desempeña el driver JDBC puesto que está situado en la base de cada uno de los marcos de trabajo.

La ventaja de usar conectores JDBC es que independiza de la base de datos que utilice.

No obstante hay un trabajo de traducción para mapear los campos devueltos por cada consulta a la colección de objetos correspondiente. Y hay que trabajar las sentencias de actualización, inserción y eliminación para cada uno de los campos. Esto constituye una razón para tratar de buscar alternativas menos costosas en tiempo de desarrollo.

El controlador de ODBC (Open Database Connectivity, Conectividad abierta de bases de datos) es la interfaz de programación de base de datos que utiliza Microsoft para tener acceso a distintas bases de datos relacionales en diversas plataformas. También existe un estándar que sirve de puente entre JDBC-ODBC en las versiones Solaris y Windows de la plataforma Java, para que se pueda utilizar ODBC desde un programa Java.

6.- Mapeo objeto relacional (ORM).

En los inicios de la programación, los programas se desarrollaban con la llamada programación imperativa. De hecho, y como sabes, la programación orientada a objetos surgió unos años después.

Con la expansión de la programación orientada a objetos, se da la circunstancia de que las bases de datos relacionales se siguen utilizando hoy en día de manera masiva, por lo que muchas aplicaciones se desarrollan con POO pero los datos, no se almacenan en bases de datos orientadas a objetos, sino en las bases de datos relacionales.

El sistema más extendido en las empresas hoy en día para guardar la información de sus aplicaciones es el uso de una base de datos relacional. Tradicionalmente, dichas aplicaciones están basadas en sentencias

SQL con las cuales se gestionan todos los datos que manejan. Este sistema es la base para el continuo crecimiento del mapeo Objeto-Relacional (O/R) y está asociado a los mecanismos de persistencia.

Cuando se programan sistemas orientados a objetos, utilizando una base de datos relacional, los programadores invierten gran cantidad de tiempo en desarrollar los objetos persistentes, o sea, convertir los objetos del lenguaje de programación a registros de la base de datos. Igualmente, también pasan bastante tiempo implementando la operación inversa, es decir, convirtiendo los registros en objetos.

El **mapeo objeto-relacional** (Object-Relational Mapping, o ORM) consisten en una técnica de programación para convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y el sistema utilizado en una base de datos relacional.

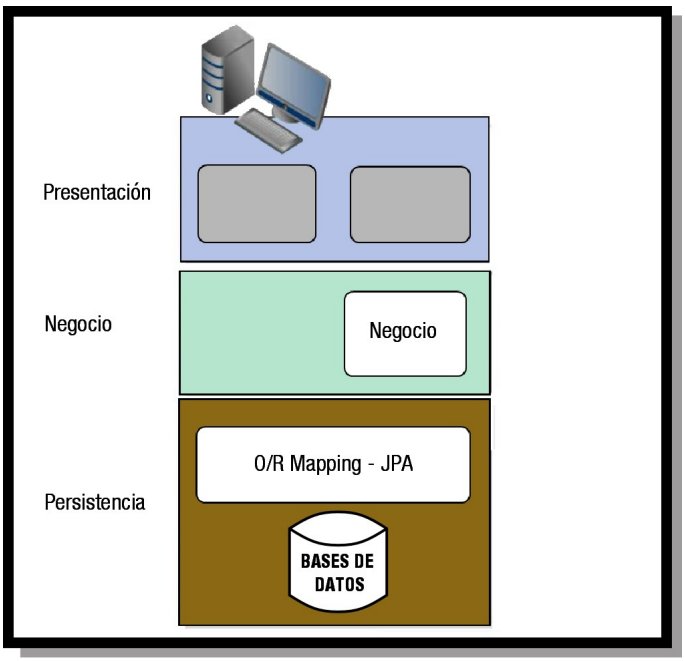
Cuando se trabajan con programación orientada a objetos y con bases de datos relacionales, es fácil observar que estos son dos paradigmas diferentes. El modelo relacional trata con relaciones y conjuntos, es de naturaleza matemática. Por el contrario, el paradigma orientado a objetos trata con objetos, atributos y asociaciones de unos con otros.

Cuando se requiere almacenar la información de los objetos utilizando una base de datos relacional se comprueba que hay un problema de compatibilidad entre estos dos paradigmas, el llamado **desfase objeto-relacional**.

Por ello, para ahorrar trabajo al programador, se puede utilizar un framework que se encargue de realizar estas tareas de modo transparente, de modo que el programador no tenga por qué usar JDBC ni SQL y la gestión del acceso a base de datos esté centralizada en un componente único permitiendo su reutilización.

6.1.- Capa de persistencia y framework de mapeo.

La **capa de persistencia** de una aplicación es la pieza que permite almacenar, recuperar, actualizar y eliminar el estado de los objetos que necesitan persistir en un sistema gestor de datos.



En el caso del mapeo objeto-relacional, un ORM es una capa que permite relacionar objetos con un modelo de datos relacional, ocultando todo el mecanismo de conexión al motor de base de datos, y también no teniendo que escribir las sentencias SQL necesarias para la gestión de los datos.

La capa de persistencia traduce entre los dos modelos de datos: desde objetos a registros y desde registros a objetos. Así, si el programa quiere grabar un objeto, entonces llama al motor de persistencia, el motor de persistencia traduce el objeto a registros y llama a la base de datos para que guarde estos registros.

De este modo el programa sólo ve que puede guardar y recuperar objetos, como si estuviera programado para una base de datos orientada a objetos. Y la base de datos sólo ve que guarda y recupera registros como si el programa estuviera dirigiéndose a ella de forma relacional.

Dispones de múltiples alternativas como desarrollador en Java cuando pretendas trabajar con mapeadores O/R. Hay tres comunidades que están implicadas en el mundo de la persistencia O/R de Java de forma activa:

- Organizaciones basadas en el estándar,
- Comunidades código abierto (open source) y
- Grupos comerciales.

Las comunidades open source incluyen importantes tecnologías, entre ellas Hibernate y el framework Spring.

Las alternativas más importantes basadas en el estándar, son EJB 3.0 y JDO.

Entre las implementaciones comerciales se puede resaltar TopLink.

Cada uno de los mecanismos de mapeo O/R tiene una dependencia particular en el conector JDBC para poder comunicarse con la base de datos de una forma eficiente. Si el conector JDBC que participa en la comunicación no es óptimo, la posible gran eficiencia de cualquier framework quedará debilitada. Por tanto, seleccionar el driver JDBC que mejor se adapte a la aplicación es esencial a la hora de construir un sistema eficiente en el que intervenga un mecanismo de mapeo O/R.

7.- Bases de datos XML

Debido a algunas limitaciones de la tecnología de bases de datos relacionales están surgiendo nuevas clases de bases de datos como son las bases de datos XML. Se define XML como: eXtensible Markup Language (lenguaje de marcado extensible). Es una especificación del World Wide Web Consortium (W3C).

Estas bases de datos almacenan los datos estructurados como XML sin la necesidad de traducir los datos a una estructura relacional o de objeto.

Podemos distinguir entre:

- **Bases de datos nativas XML**. Consiste en un modelo lógico para documentos XML. El Sistema Gestor de Base de Datos correspondiente almacena y recupera documentos de acuerdo a dicho modelo.
  - Productos ejemplo de esa filosofía son: **eXist**, Apache Xindice, Berkeley dbXML.…
- **Bases de datos compatibles con XML (XML-enabled)**: son bases de datos, generalmente objeto-relacionales, que admiten entradas de datos en forma de XML y proporcionan salidas en este mismo formato.
  - Podemos citar productos como: Oracle, DB2 (Viper),...

Las bases de datos XML nativas permiten trabajar con XQL (eXtensible Query Language), el cuál sirve un propósito similar a SQL en una base de datos relacional.

El trabajo con bases de datos XML nativas involucra dos pasos básicos:

- Describir los datos mediante Definiciones de Tipos de Datos (Document Type Definitions, DTD) o esquemas XML y
- Definir un nuevo esquema de base de datos XML nativa o Mapa de Datos a usar para almacenar y obtener datos.

Bases de datos XML
https://db-engines.com/en/ranking/native+xml+dbms

Inconvenientes:

- Son más lentas y requieren que los datos estén comprimidos para funcionar más rápidamente.
- Las búsquedas son más lentas que en una base de datos relacional, ya que se deben organizar a través de texto y etiquetas.
- Existe cierta limitación en cuanto a los gestores de bases de datos que pueden utilizar lenguaje XML.
- Las bases de datos creadas con documentos XML no están preparadas para el almacenamiento de información a largo plazo.
- Pueden existir problemas para garantizar la seguridad de los datos. Por ejemplo, no se pueden configurar para definir quién puede actualizar, añadir o eliminar la información de la base de datos.
- No son muy utilizadas en el sector profesional

8.- Bases de datos Documentales

El uso de los sistemas de bases Documentales permite superar los problemas de escalabilidad y rendimiento que se producen en los sistemas de bases de datos relacionales cuando se dan cita una gran cantidad de usuarios concurrentes y millones de consultas diarias. Estos sistemas de bases de datos están optimizados para las tareas de recuperación y agregación de información.

Además, las bases de datos Documentales son sistemas de almacenamiento de información que no cumplen el esquema Entidad-Relación, ni la estructura de tabla en la que se almacenan los datos. Estos sistemas almacenan la información haciendo uso de formatos como **Clave-Valor, mapeo de columnas o grafos**.

La mayor parte de los sistemas de bases de datos NoSQL no utilizan este lenguaje o simplemente como apoyo. Por ejemplo, **MongoDB** utiliza **JSON**, BigTable utiliza GQL y Cassandra utiliza CQL.

MongoDB utiliza un lenguaje propio para el almacenamiento de la información, BSON que es una evolución de JSON, con la particularidad de poder almacenar datos binarios. Actualmente, MongoDB es la base de datos preferida entre los desarrolladores.

Puedes visitar https://www.mongodb.com/es para más información.

Algunas empresas que utilizan este tipo de bases de datos

Estas son algunas de las grandes empresas de Internet que utilizan este tipo de bases de datos.

- **Cassandra**: Twitter, Facebook…
- **MongoDB**: FourSquare, SourceForge…
- **HBase**: Yahoo, Adobe…
- **Redis**: Flickr, Instagram, Github…
- **Neo4j**: Infojobs…

Bases de datos Documentales
https://db-engines.com/en/ranking/document+store

9.- Desarrollo de componentes

La expansión, en los últimos años, de Internet y el comercio electrónico ha llevado a la necesidad de adoptar nuevas tecnologías y nuevos requisitos de desarrollo en las aplicaciones informáticas.

Al principio de la informática, los ordenadores eran caros y la mano de obra de los programadores asequible. Con el paso del tiempo los ordenadores son baratos y la mano de obra de los programadores suele ser cara. Además, los requisitos de las aplicaciones informáticas son cada vez más complejos. Por ello, aparecieron las técnicas orientadas a objetos, para, entre otras cosas, intentar programar de manera que el código que se desarrolle pueda ser reutilizable.

9.1.- Definición de componente

Un **componente** es una unidad de software que realiza una función bien definida y posee una interfaz bien definida.

Un componente software posee interfaces especificadas contractualmente, pudiendo ser desplegado independientemente y puede interaccionar con otros componentes para formar un sistema.

Un interfaz es un punto de acceso a los componentes: permite a los clientes acceder a los servicios proporcionados por un componente.

La idea de dividir u organizar en trozos el software, o sea, dividirlo en componentes surge para reducir la complejidad del software. Así se permite la reutilización y se acelera el proceso de ensamblaje del software.

Los creadores de componentes pueden especializarse creando objetos cada vez mas complejos y de mayor calidad.

La interoperabilidad entre componentes de distintos fabricantes:

- Aumenta la competencia,
- Reduce los costes y,
- Facilita la construcción de estándares.

Por tanto, así el software se hace cada vez más rápido, de mejor calidad y a menor coste incluso de mantenimiento.

Un componente software también debe especificar sus necesidades para funcionar correctamente, lo que se denominan las dependencias de contexto:

- Interfaces requeridas
- Entorno de ejecución: máquina necesaria, sistema operativo a utilizar, etc.

En el entorno Java, contamos con los JavaBeans.

9.2.- JavaBeans

El origen de los JavaBeans lo podemos encontrar en un par de necesidades que Java tenía:

- Disponer de una tecnología de objetos y componentes reutilizables.
- Mejorar el proceso para crear interfaces de usuario, acercándose a la facilidad de uso que otros productos permitían como Visual Basic de Microsoft.

Un **JavaBean** es un componente software reutilizable basado en la especificación JavaBean de Sun (ahora Oracle) que se puede manipular visualmente con una herramienta de desarrollo.

Hay una serie de propiedades que presenta un JavaBean:

- **Portabilidad**: uno de los principales objetivos de los JavaBeans es proporcionar una arquitectura neutral de componentes, es decir, que los beans puedan utilizarse en otras plataformas y entornos.
- **Reusabilidad**: son componentes reutilizables, la filosofía es que estos componentes pueden usarse como bloques en la construcción de aplicaciones complejas.
- **Introspección**: los IDE reconocen ciertas pautas de diseño, nombres de las funciones miembros o métodos y definiciones de las clases, permitiendo a la herramienta de programación mirar dentro del bean y conocer sus propiedades y comportamiento.
- **Personalización**: en tiempo de diseño, con el IDE que se utilice, se pueden modificar las características de apariencia y comportamiento de un bean.
- **Persistencia**: un bean puede guardar su estado y recuperarlo posteriormente. Esta capacidad se de logra mediante la serialización. Cuando un ejemplar de bean se serializa se convierte en un flujo de datos que se almacenará en algún sitio, probablemente en un fichero. Cualquier applet, aplicación o herramienta que utilice el bean puede restaurarlo mediante la deserialización.

- **Comunicación entre eventos**: Los eventos constituyen un mecanismo de notificación entre objeto fuente y objeto(s) receptor(es). Las herramientas de desarrollo pueden examinar un bean para determinar qué eventos puede enviar y cuáles puede recibir.

Características de los JavaBean

Las reglas básicas para diseñar un JavaBean:

- Se define como clase pública.
- Debe tener un constructor vacío por defecto, aunque puede tener más constructores.
- Se debe implementar el interface Serializable para poder implementar la persistencia.
- Se deben seguir las convenciones de nombres establecidas, como por ejemplo los metodos get...() y set...() se usan para obtener y establecer propiedades de un Bean.
- Una propiedad es de solo lectura o solo escritura si falta uno de los métodos de acceso anteriores.

```
public class MiJavaBean implents Serializable {

     String miPropiedad;

     // Constructor
     public MiJavaBean() {
          miPropiedad = "";
     }

     // Getters y Setters
     public void setMiPropiedad(String s) {
          miPropiedad = s;
     }

     public String getMiPropiedad() {
          return miPropiedad;
     }

     // Métodos añadidos
     public void miMetodo() {
          System.out.println("Ejemplo de Javabean");
     }

}
```

Fuente: ***MECD***